Faculté des Sciences *Semlalia* de Marrakech
Département de l'Informatique

Rapport de :

MINI PROJET EN C++

Sujet :

APPLICATION DE JEU DE SFRCH

Encadré par :

Prof : RACHIDA HANNANE

Réalisé par :
Soufiane Oushi
Apogé :2418141
&
Rachid Yaaqoub
Apogé : 2331082

« Groupe 4 SINF4 »

©Année Universitaire 2025-2026©

Table des matières

1. **Introduction à la Programmation Orientée Objet en C++**
    - 1.1 Qu'est-ce que la Programmation Orientée Objet ?
    - 1.2 Les quatre piliers fondamentaux de la POO
    - 1.3 Concepts C++ avancés utilisés dans le projet
2. **Création d'un Jeu Graphique avec SFML**
    - 2.1 Présentation de la bibliothèque SFML
    - 2.2 La Game Loop SFML — Cœur du jeu
    - 2.3 Défilement de l'arrière-plan avec ArrierePlan
    - 2.4 Formes géométriques SFML utilisées dans le projet
    - 2.5 Gestion du texte et de la police
3. **Idée Générale du Jeu — Description & Gameplay**
    - 3.1 Contexte et univers du jeu
    - 3.2 Règles du jeu
    - 3.3 Architecture des niveaux
    - 3.4 Contrôles et interface utilisateur
    - 3.5 États du jeu (EtatJeu)
4. **Identification des Classes — Rôles, Attributs, Méthodes**
    - 4.1 Constantes globales — constantes.h
    - 4.2 Classe Joueur
    - 4.3 Classe Obstacle
    - 4.4 Classe ArrierePlan
    - 4.5 Classe TexturesObstacles
    - 4.6 Classe Jeu
5. **Diagramme de Classes UML**
    - 5.1 Diagramme UML complet
    - 5.2 Tableau des relations entre classes
6. **Fonctions Principales — Description Détaillée**
    - 6.1 Jeu::lancer()
    - 6.2 Jeu::gererEvenements()
    - 6.3 Jeu::mettreAJour()
    - 6.4 Joueur::sauter() et Joueur::sAccroupir()
    - 6.5 Jeu::apparaitreObstacle()
    - 6.6 Jeu::afficher()
    - 6.7 Jeu::dessinerJeu()
    - 6.8 ArrierePlan::reinitialiser()
7. **Besoins de Compilation**
    - 7.1 Prérequis logiciels
    - 7.2 Structure complète des fichiers du projet
    - 7.3 Installation de SFML selon la plateforme
    - 7.4 Compilation manuelle avec g++
    - 7.5 Compilation avec CMake
    - 7.6 Drapeaux de compilation — Référence
    - 7.7 Erreurs fréquentes et solutions
    - 7.8 References
8. **Conclusion**

1. Introduction à la Programmation Orientée Objet en C++

1.1 Qu'est-ce que la Programmation Orientée Objet ?

La Programmation Orientée Objet (POO) est un paradigme de développement logiciel fondé sur le concept d'« objet » : une entité autonome qui regroupe à la fois des données (attributs) et des comportements (méthodes). Ce paradigme constitue aujourd'hui le modèle dominant dans le développement de logiciels professionnels, et tout particulièrement dans le développement de jeux vidéo.

Le C++ est un langage dit multi-paradigme : il supporte la programmation procédurale, générique et orientée objet dans un même langage. C'est pourquoi il est si largement utilisé dans l'industrie du jeu vidéo (Unreal Engine, par exemple, est entièrement écrit en C++).

Dans notre projet, la POO nous a permis de modéliser chaque entité du jeu (joueur, obstacle, arrière-plan) sous forme de classe indépendante, avec ses propres données et comportements, tout en permettant à la classe **Jeu** d'orchestrer l'ensemble.

1.2 Les quatre piliers fondamentaux de la POO

Pilier 1 --- Encapsulation

L'encapsulation consiste à regrouper les données et les traitements au sein d'une même classe, tout en contrôlant leur accès de l'extérieur. En C++, cela se fait grâce aux modificateurs d'accès **private**, **protected** et **public**.

Dans la classe **Joueur**, les attributs **vies**, **etat** et **invincible** ne sont modifiables que via des méthodes dédiées comme **recevoirCoup()** ou **reinitialiser()**. Cela garantit la cohérence de l'état du joueur à tout moment.

Joueur.h --- attributs et méthodes de modification

```
int   vies       = 3;
bool  invincible = false;
Etat  etat       = Etat::COURIR;
float vitesseY   = 0.f;
bool  surSol     = true;

bool recevoirCoup();   // seul moyen de perdre une vie
void reinitialiser();  // réinitialise tout l'état du joueur
void sauter();
void sAccroupir(bool actif);
```

Pilier 2 --- Héritage

L'héritage permet à une classe dérivée de réutiliser le code d'une classe parente, en héritant de ses attributs et méthodes. Dans notre projet actuel, l'héritage n'est pas directement implémenté pour simplifier la conception, mais la structure est conçue pour l'accueillir naturellement.

Extension possible par héritage (C++ abstrait)

```
class Entite {
protected:
    float posX, posY;
    int   vies;
public:
    virtual void dessiner(sf::RenderWindow& w) const = 0;
    virtual bool estMort() const = 0;
    virtual sf::FloatRect obtenirBornes() const = 0;
};

class Joueur  : public Entite { /* ... */ };
class Obstacle : public Entite { /* ... */ };
```

Pilier 3 --- Polymorphisme

Le polymorphisme signifie « plusieurs formes » : un même appel de méthode peut produire des comportements différents selon le type réel de l'objet. En C++, il se manifeste via la surcharge de fonctions (compile-time) et les fonctions virtuelles (run-time).

Polymorphisme dynamique --- dessiner() sur toutes les entités

```
std::vector<Entite*> entites;
entites.push_back(new Joueur());
entites.push_back(new Obstacle(x, false, vitesse, tex, true));

for (auto* e : entites) {
    e->dessiner(window); // appelle la bonne méthode selon le type réel
}
```

Pilier 4 --- Abstraction

L'abstraction consiste à ne présenter à l'utilisateur d'une classe que l'interface nécessaire, en cachant les détails internes. Les fichiers .h (headers) de notre projet jouent précisément ce rôle : ils exposent l'interface publique d'une classe sans révéler comment elle fonctionne en interne.

1.3 Concepts C++ avancés utilisés dans le projet

Constructeurs et règle des trois

Chaque classe possède un constructeur qui initialise ses attributs de façon cohérente. La classe **Joueur** définit explicitement un constructeur de copie et un opérateur d'affectation car elle contient un **sf::Sprite** qui référence une **sf::Texture** --- sans cela, la copie laisserait un pointeur dangling.

Joueur.cpp --- constructeur de copie (règle des trois)

```
Joueur::Joueur(const Joueur& o)
    : texture(o.texture), utiliserSecours(o.utiliserSecours),
      vitesseY(o.vitesseY), surSol(o.surSol),
      etat(o.etat), vies(o.vies), invincible(o.invincible)
{
    sprite.setTexture(texture);  // pointe vers notre propre copie
    sprite.setScale(o.sprite.getScale());
    sprite.setPosition(o.sprite.getPosition());
    sprite.setColor(o.sprite.getColor());
    rectSecours = o.rectSecours;
}
```

Namespace et constantes globales

constantes.h --- namespace Constantes (extrait)

```
namespace Constantes {
    const int   LARGEUR_FENETRE   = 900;
    const int   HAUTEUR_FENETRE   = 500;
    const float SOL_Y             = 380.f;
    const float JOUEUR_L          = 70.f;
    const float JOUEUR_H          = 90.f;
    const float PESANTEUR         = 0.5f;
    const float FORCE_SAUT        = -15.f;
    const int   NB_NIVEAUX        = 3;
    const float VITESSE_OBS[3]    = { 4.f, 6.f, 8.f };
}
```

Enum class --- type énuméré fortement typé

EtatJeu.h et Joueur.h --- enum class

```
// États de la machine à états du jeu
enum class EtatJeu {
    MENU, A_PROPOS, INTRO_NIVEAU, JOUER,
    NIVEAU_REUSSI, FIN_PARTIE, VICTOIRE
};

// États internes du joueur (imbriqué dans Joueur)
enum class Etat { COURIR, SAUTER, ACCROUPI, BLESSE, MORT };

if (etat == EtatJeu::JOUER) { mettreAJour(); }
if (joueur.estMort())       { etat = EtatJeu::FIN_PARTIE; }
```

2. Création d'un Jeu Graphique avec SFML

2.1 Présentation de la bibliothèque SFML

SFML (Simple and Fast Multimedia Library) est une bibliothèque C++ libre, multi-plateforme, développée par Laurent Gomila. Elle fournit une couche d'abstraction haut niveau au-dessus d'OpenGL, DirectX et des API système.

Module SFML	Rôle
sf::System	Horloges, vecteurs, threads (sf::Clock, sf::Vector2f)
sf::Window	Fenêtre système, événements clavier/souris
sf::Graphics	Rendu 2D, formes, textures, polices --- module principal du projet
sf::Audio	Lecture de sons et musiques
sf::Network	Communication réseau TCP/UDP

2.2 La Game Loop SFML --- Cœur du jeu

Tout jeu vidéo repose sur une boucle principale (game loop) qui tourne en continu tant que le jeu est ouvert. Cette boucle effectue à chaque itération exactement trois étapes : gestion des événements, mise à jour (Update), et rendu (Draw).

Architecture de la Game Loop dans Jeu::lancer()

```
void Jeu::lancer() {
    while (fenetre.isOpen()) {
        gererEvenements();  // étape 1 : événements
        mettreAJour();      // étape 2 : logique
        afficher();         // étape 3 : rendu
    }
}
```

2.3 Défilement de l'arrière-plan avec ArrierePlan

Le défilement infini de l'arrière-plan est géré par la classe **ArrierePlan**. Elle maintient deux tuiles (sprites) côte à côte. À chaque frame, elles se décalent vers la gauche à la vitesse des obstacles. Quand une tuile sort de l'écran, elle est repositionnée juste après l'autre, créant l'illusion d'un fond continu.

ArrierePlan::defiler() --- défilement cyclique à deux tuiles

```
void ArrierePlan::defiler(float vitesse) {
    if (!charge) return;
    for (int i = 0; i < 2; ++i) {
        tuiles[i].move(-vitesse, 0.f);
        if (tuiles[i].getPosition().x + Constantes::LARGEUR_FENETRE <= 0.f) {
            float autreX = tuiles[1-i].getPosition().x;
            tuiles[i].setPosition(autreX + Constantes::LARGEUR_FENETRE, 0.f);
        }
    }
}
```

2.4 Formes géométriques SFML utilisées dans le projet

Joueur::dessiner() --- rendu SFML avec sprite ou secours

```
void Joueur::dessiner(sf::RenderWindow& fenetre) const {
    if (utiliserSecours) {
        fenetre.draw(rectSecours);         // rectangle bleu de remplacement
        sf::CircleShape tete(JOUEUR_L * 0.35f);
        tete.setFillColor(sf::Color(100, 170, 255, 220));
        fenetre.draw(tete);
    } else {
        fenetre.draw(sprite);              // sprite PNG chargé
    }
}
```

2.5 Gestion du texte et de la police

Jeu --- chargement de la police dans le constructeur

```
if (!police.loadFromFile("arial.ttf")) {
    std::cerr << "[ERREUR] Police arial.ttf introuvable\n";
}

// Utilisation typique pour afficher du texte centré
sf::Text texte("NIVEAU 1 - FORET", police, 28);
texte.setFillColor(sf::Color::White);
sf::FloatRect b = texte.getLocalBounds();
texte.setOrigin(b.left + b.width/2.f, b.top + b.height/2.f);
texte.setPosition(LARGEUR_FENETRE/2.f, HAUTEUR_FENETRE/2.f);
fenetre.draw(texte);
```

3. Idée Générale du Jeu --- Description & Gameplay

3.1 Contexte et univers du jeu

Le jeu est un **runner 2D en vue de côté**. Le joueur incarne un personnage qui court automatiquement et doit éviter des obstacles en sautant ou en s'accroupissant. Le jeu est structuré en **3 niveaux** aux thèmes visuels distincts : Forêt, Kalahari et Arctique. L'esthétique est délibérément minimaliste, axée sur la lisibilité du gameplay.

3.2 Règles du jeu

Le joueur dispose de **3 vies**. Chaque collision avec un obstacle lui en retire une. Après une collision, une période d'**invincibilité temporaire** de 2,5 secondes est activée (clignotement orange), évitant les pénalités consécutives injustes. La perte des 3 vies déclenche la fin de partie.

Il existe deux catégories d'obstacles : les obstacles **au sol** (à éviter en sautant) et les obstacles **aériens** (à éviter en s'accroupissant). La méthode **obstacleAerienDevant()** dans la classe **Jeu** vérifie automatiquement la présence d'un obstacle aérien avant d'autoriser le saut.

Niveau	Détails

Niveau 1 --- FORET	Vitesse 4→8 px/frame | Durée 150s | Distance 2500 | Intervalle min 1.2s
Niveau 2 --- KALAHARI	Vitesse 6→11 px/frame | Durée 100s | Distance 3000 | Intervalle min 0.9s
Niveau 3 --- ARCTIQUE	Vitesse 8→15 px/frame | Durée 90s | Distance 7000 | Intervalle min 0.6s

3.3 Architecture des niveaux

Chaque niveau est défini par ses paramètres dans le namespace **Constantes** : vitesse initiale et maximale des obstacles, durée en secondes, distance à parcourir et intervalle minimal entre apparitions. La classe **Jeu** utilise l'attribut **niveauActuel** (entier 0-2) pour indexer ces tableaux de paramètres.

3.4 Contrôles et interface utilisateur

Touche(s)	Action
ESPACE / Flèche Haut	Sauter --- impossible si accroupi ou obstacle aérien détecté devant
Flèche Bas	S'accroupir --- permet de passer sous les obstacles aériens
ESCAPE	Quitter le jeu / Retour au menu

3.5 États du jeu (EtatJeu)

Valeur	Description	Contexte d'utilisation
MENU	Menu principal	Écran d'accueil, lancement de partie
A_PROPOS	À propos du jeu	Affiche les crédits et informations
INTRO_NIVEAU	Introduction d'un niveau	Texte de présentation avant de jouer
JOUER	Partie en cours	Boucle principale du jeu, gameplay actif
NIVEAU_REUSSI	Niveau terminé avec succès	Écran de résultats, passage au niveau suivant
FIN_PARTIE	Fin de partie (échec)	Game over, proposition de recommencer
VICTOIRE	Victoire globale	Tous les niveaux réussis, écran de fin

4. Identification des Classes --- Rôles, Attributs, Méthodes

4.1 Constantes globales --- constantes.h

Le fichier **constantes.h** centralise toutes les valeurs de configuration du jeu dans le namespace **Constantes**. Ce choix garantit la lisibilité et évite les macros #define

non typées. L'ensemble des classes inclut ce fichier pour accéder aux constantes via le préfixe **Constantes::**.

constantes.h --- code complet (extrait)

```
#ifndef CONSTANTES_H
#define CONSTANTES_H
#include <string>

namespace Constantes {
    const int   LARGEUR_FENETRE        = 900;
    const int   HAUTEUR_FENETRE        = 500;
    const float SOL_Y                  = 380.f;
    const float JOUEUR_L               = 70.f;
    const float JOUEUR_H               = 90.f;
    const float JOUEUR_X               = 100.f;
    const float PESANTEUR              = 0.5f;
    const float FORCE_SAUT             = -15.f;
    const float ACCROUPISSEMENT_H      = 45.f;
    const float OBS_L                  = 90.f;
    const float OBS_H_SOL              = 90.f;
    const float OBS_H_AIR              = 100.f;
    const int   NB_NIVEAUX             = 3;
    const float VITESSE_OBS[3]         = { 4.f,  6.f,  8.f  };
    const float VITESSE_MAX_OBS[3]     = { 8.f, 11.f, 15.f  };
    const int   DUREE_NIVEAU[3]        = { 150,  100,   90   };
    const float DISTANCE_NIVEAU[3]     = { 2500.f, 3000.f, 7000.f };
    const float INTERVALLE_MIN[3]      = { 1.2f,   0.9f,   0.6f  };
}
#endif
```

4.2 Classe Joueur

Rôle & Responsabilité	Représente le personnage contrôlé par l'utilisateur. Gère la physique (gravité, saut, accroupissement), les états internes, les vies et le rendu (sprite PNG ou rectangle de secours si la texture est absente).
Attributs	sprite : sf::Sprite | texture : sf::Texture rectSecours : sf::RectangleShape | utiliserSecours : bool vitesseY : float --- composante verticale de déplacement surSol : bool --- vrai si le joueur repose sur le sol etat : Etat (COURIR, SAUTER, ACCROUPI, BLESSE, MORT) vies : int --- initialisé à 3 invincible : bool --- vrai pendant 2.5s après un coup horlogeBlessure : sf::Clock
Constructeur	Joueur() --- tente de charger 'perso.png' et filtre les pixels noirs/blancs pour la transparence. En cas d'échec, active le mode secours (rectangle bleu). Définit également le constructeur de copie et l'opérateur= (règle des trois).
Méthodes publiques	sauter(), sAccroupir(bool actif), mettreAJour() recevoirCoup() → bool (retourne vrai si le joueur est mort)

	obtenirBornes() → sf::FloatRect (hitbox réduite à 80%×90%) dessiner(RenderWindow&) estMort(), estAccroupi(), estEnAir()
Relations	← Jeu l'agrège par valeur (composition 1:1) ← Obstacle l'utilise indirectement via Jeu pour la collision

Joueur::recevoirCoup() --- gestion des vies et invincibilité

```
bool Joueur::recevoirCoup() {
    if (invincible) return false;
    vies--;
    invincible = true;
    horlogeBlessure.restart();
    if (vies <= 0) {
        etat = Etat::MORT;
        sprite.setColor(sf::Color(180, 40, 40));
        return true;   // signal : le joueur est mort
    }
    etat = Etat::BLESSE;
    return false;
}
```

4.3 Classe Obstacle

Rôle & Responsabilité	Représente un obstacle mobile (au sol ou aérien) se déplaçant de droite à gauche. Peut utiliser une texture PNG ou un rectangle coloré de secours. Signale sa sortie d'écran pour être supprimé par Jeu.
Attributs	estAerien : bool --- détermine la position Y et la hauteur sprite : sf::Sprite | rectSecours : sf::RectangleShape utiliserSecours : bool _vitesse : float --- vitesse de déplacement horizontal _detruit : bool --- marqué pour suppression par Jeu
Constructeur	Obstacle(x, aerien, vitesse, tex, texChargee) Se positionne automatiquement : OBS_SOL_Y (sol) ou OBS_AIR_Y (aérien) Dimensionne le sprite à OBS_L × OBS_H_SOL ou OBS_H_AIR
Méthodes publiques	deplacer() --- décale de _vitesse pixels vers la gauche dessiner(RenderWindow&) estHorsEcran() → bool obtenirBornes() → sf::FloatRect obtenirX() → float detruire(), estDetruit()
Relations	← Jeu maintient un vector<Obstacle> et les met à jour ← Jeu reçoit une référence sf::Texture de TexturesObstacles

4.4 Classe ArrierePlan

Rôle & Responsabilité	Gère le défilement infini du fond d'écran via deux tuiles (sprites) recyclées cycliquement. Une instance par niveau --- la classe Jeu en maintient un tableau de 3.
Attributs	texture : sf::Texture --- texture partagée par les deux tuiles tuiles[2] : sf::Sprite --- les deux tuiles côte à côte charge : bool --- vrai si le chargement a réussi
Constructeur	Pas de constructeur explicite. L'initialisation se fait via charger() qui dimensionne les tuiles à la taille de la fenêtre.
Méthodes publiques	charger(fichier) → bool --- charge la texture et dimensionne les tuiles defiler(vitesse) --- déplace et recycle les tuiles dessiner(RenderWindow&) reinitialiser() --- remet les tuiles en position initiale
Relations	← Jeu possède arrieresPlans[3] (un par niveau) ← Jeu appelle defiler() à chaque frame avec la vitesse courante

ArrierePlan::charger() --- mise à l'échelle automatique

```
bool ArrierePlan::charger(const std::string& fichier) {
    if (!texture.loadFromFile(fichier)) return false;
    texture.setSmooth(true);
    float sx = (float)Constantes::LARGEUR_FENETRE / texture.getSize().x;
    float sy = (float)Constantes::HAUTEUR_FENETRE / texture.getSize().y;
    for (int i = 0; i < 2; ++i) {
        tuiles[i].setTexture(texture);
        tuiles[i].setScale(sx, sy);
        tuiles[i].setPosition(i * (float)Constantes::LARGEUR_FENETRE, 0.f);
    }
    charge = true;
    return true;
}
```

4.5 Classe TexturesObstacles

Rôle & Responsabilité	Centralise le chargement de toutes les textures d'obstacles. Évite les chargements multiples et fournit à chaque Obstacle une référence vers la texture appropriée selon le niveau et le type (sol/aérien).
Attributs	sol[3] : sf::Texture --- textures obstacles au sol (une par niveau) aérien[3] : sf::Texture --- textures obstacles aériens (une par niveau) solCharge[3] : bool[3] --- état de chargement sol aerienCharge[3] : bool[3] --- état de chargement aérien
Méthodes publiques	charger() --- itère sur les 3 niveaux, appelle loadFromFile() pour chaque texture, affiche un avertissement si échec
Relations	← Jeu en possède une instance (texObs) ← Jeu passe tex.sol[n] ou tex.aerien[n] au constructeur d'Obstacle

4.6 Classe Jeu

Rôle & Responsabilité	Classe principale qui orchestre l'ensemble du jeu : gestion de la fenêtre SFML, machine à états (EtatJeu), boucle principale, génération et mise à jour des obstacles, détection des collisions, et rendu de tous les éléments.
Attributs principaux	fenetre : sf::RenderWindow (900×500) police : sf::Font etat : EtatJeu | niveauActuel : int (0-2) joueur : Joueur | texObs : TexturesObstacles obstacles : vector<Obstacle> arrieresPlans[3] : ArrierePlan horlogeJeu, horlogeApparition, horlogeIntro : sf::Clock distanceParcourue : float sequenceObstacles : vector<bool> (séquence prédéfinie)
Méthodes publiques	Jeu() --- crée la fenêtre 900×500, charge la police et les textures lancer() --- boucle principale gererEvenements() | mettreAJour() | afficher()
Méthodes privées	demarrerNiveau(n), lancerPartie() apparaitreObstacle() --- suit la séquence prédéfinie obstacleAerienDevant() → bool dessinerFondNiveau(vitesse) dessinerMenu / APropos / IntroNiveau / Jeu / NiveauReussi dessinerFinPartie / Victoire afficherTextreCentre(texte, taille, y, couleur)
Relations	← Jeu agrège Joueur (composition 1:1) ← Jeu agrège ArrierePlan[3] et TexturesObstacles ← Jeu gère vector<Obstacle> (création/destruction)

5. Diagramme de Classes UML

5.1 Diagramme UML complet

Le diagramme suivant représente l'ensemble des classes du projet avec leurs attributs, méthodes et relations. Les relations de composition indiquent que l'objet contenu est détruit avec le conteneur.

Diagramme UML --- Vue d'ensemble

```
┌──────────────────────────────────────────────────────────┐
│ <<namespace>> Constantes (constantes.h)                  │
│  LARGEUR_FENETRE=900  HAUTEUR_FENETRE=500                 │
│  SOL_Y=380  PESANTEUR=0.5  FORCE_SAUT=-15                 │
│  NB_NIVEAUX=3  VITESSE_OBS[3]  DISTANCE[3] ...            │
└──────────────────────────────────────────────────────────┘
                        │
                        │ utilise
┌──────────────────────▼───────────────────────────────────┐
│ Jeu                                                      │
│ - fenetre : sf::RenderWindow                             │
│ - etat : EtatJeu  - niveauActuel : int                   │
│ - joueur : Joueur                                        │
│ - arrieresPlans[3] : ArrierePlan                         │
│ - texObs : TexturesObstacles                             │
│ - obstacles : vector<Obstacle>                           │
│ + lancer()  + gererEvenements()  + afficher()            │
└──────────────────────────────────────────────────────────┘
        │                │                │
   comp │           comp │           comp │
┌───────▼──────┐  ┌──────▼──────────┐  ┌▼──────────────────┐
│ Joueur       │  │ ArrierePlan     │  │ TexturesObstacles  │
│ - vies : 3   │  │ - tuiles[2]     │  │ - sol[3]           │
│ - etat       │  │ - texture       │  │ - aerien[3]        │
│ sauter()     │  │ charger()       │  │ charger()          │
│ dessiner()   │  │ defiler()       │  └────────────────────┘
└──────────────┘  └─────────────────┘           │
                                                │ fournit texture
                                          ┌─────▼──────────────┐
                                          │ Obstacle           │
                                          │ - estAerien : bool │
                                          │ - _vitesse : float │
                                          │ deplacer()         │
                                          │ dessiner()         │
                                          └────────────────────┘
```

5.2 Tableau des relations entre classes

Relation	Description
Jeu → Joueur (Composition)	Jeu agrège joueur par valeur. Joueur est créé dans le constructeur de Jeu et détruit avec lui.
Jeu → ArrierePlan[3] (Composition)	Jeu possède trois arrière-plans (un par niveau). Réinitialisés à chaque démarrage de niveau.
Jeu → TexturesObstacles (Composition)	Jeu possède une instance texObs centralisée. Les textures sont chargées une seule fois au démarrage.
Jeu → vector<Obstacle> (Agrégation externe)	Jeu crée et détruit les obstacles selon la séquence de jeu et les nettoie quand ils sortent de l'écran.
Obstacle ← TexturesObstacles (Dépendance)	Obstacle reçoit une référence sf::Texture et un booléen texChargee lors de sa construction.
* → Constantes (Utilisation)	Toutes les classes utilisent les constantes du namespace Constantes via #include "constantes.h".

6. Fonctions Principales --- Description Détaillée

6.1 Jeu::lancer()

Méthode centrale du programme. Elle entre dans la boucle principale qui tourne tant que la fenêtre est ouverte, et délègue à **gererEvenements()**, **mettreAJour()** et **afficher()**. La machine à états **etat** conditionne les actions effectuées à chaque étape.

6.2 Jeu::gererEvenements()

Traite les événements SFML selon l'état courant. En mode **JOUER**, elle écoute ESPACE (saut) et la flèche bas (accroupissement). Dans les autres états, elle gère les transitions de menu. La touche ESCAPE provoque une transition vers MENU ou ferme la fenêtre.

Gestion du saut dans gererEvenements()

```
if (event.type == sf::Event::KeyPressed) {
    if (event.key.code == sf::Keyboard::Space ||
        event.key.code == sf::Keyboard::Up) {
        if (!obstacleAerienDevant())  // vérifie avant d'autoriser
            joueur.sauter();
    }
}
// Accroupissement : isKeyPressed (maintenu)
joueur.sAccroupir(
    sf::Keyboard::isKeyPressed(sf::Keyboard::Down));
```

6.3 Jeu::mettreAJour()

Jeu::mettreAJour() --- logique principale (état JOUER)

```
void Jeu::mettreAJour() {
    if (etat != EtatJeu::JOUER) return;
    joueur.mettreAJour();
    float vit = vitesseObs();
    arrieresPlans[niveauActuel].defiler(vit);
    for (auto& obs : obstacles) obs.deplacer();
    // Détection de collisions
    for (auto& obs : obstacles) {
        if (!obs.estDetruit() &&
            joueur.obtenirBornes().intersects(obs.obtenirBornes())) {
            if (joueur.recevoirCoup())
                etat = EtatJeu::FIN_PARTIE;
            obs.detruire();
        }
    }
    // Nettoyage des obstacles hors écran
    obstacles.erase(std::remove_if(obstacles.begin(), obstacles.end(),
        [](const Obstacle& o){ return o.estHorsEcran(); }),
        obstacles.end());
}
```

6.4 Joueur::sauter() et Joueur::sAccroupir()

Le saut n'est autorisé que si le joueur est au sol et non accroupi. L'accroupissement modifie la hauteur et la position du sprite pour permettre de passer sous les obstacles aériens. La transition retour vers COURIR est automatique à la levée de la touche.

Joueur::sauter() et Joueur::sAccroupir(bool actif)

```
void Joueur::sauter() {
    if (surSol && etat != Etat::ACCROUPI) {
        vitesseY = Constantes::FORCE_SAUT;  // -15.f
        surSol   = false;
        etat     = Etat::SAUTER;
    }
}

void Joueur::sAccroupir(bool actif) {
    if (actif && surSol && etat != Etat::ACCROUPI) {
        sprite.setScale(JOUEUR_L/tw, ACCROUPISSEMENT_H/th);
        etat = Etat::ACCROUPI;
    } else if (!actif && etat == Etat::ACCROUPI) {
        sprite.setScale(JOUEUR_L/tw, JOUEUR_H/th);
        etat = surSol ? Etat::COURIR : Etat::SAUTER;
    }
}
```

6.5 Jeu::apparaitreObstacle()

La génération des obstacles suit une **séquence prédéfinie** (vector<bool> : false = sol, true = aérien), ce qui permet de contrôler précisément la difficulté et la jouabilité. La vitesse de l'obstacle créé est calculée dynamiquement en fonction de la distance parcourue.

6.6 Jeu::afficher()

Jeu::afficher() --- ordre de rendu

```
void Jeu::afficher() {
    fenetre.clear(sf::Color(20, 20, 35));
    switch (etat) {
        case EtatJeu::MENU:          dessinerMenu();         break;
        case EtatJeu::A_PROPOS:      dessinerAPropos();      break;
        case EtatJeu::INTRO_NIVEAU:  dessinerIntroNiveau();  break;
        case EtatJeu::JOUER:         dessinerJeu();          break;
        case EtatJeu::NIVEAU_REUSSI: dessinerNiveauReussi(); break;
        case EtatJeu::FIN_PARTIE:    dessinerFinPartie();    break;
        case EtatJeu::VICTOIRE:      dessinerVictoire();     break;
    }
    fenetre.display();
}
```

6.7 Jeu::dessinerJeu()

Ordre d'affichage dans dessinerJeu()

```
void Jeu::dessinerJeu() {
    arrieresPlans[niveauActuel].dessiner(fenetre); // 1. fond défilant
    fenetre.draw(rectSol);                         // 2. sol
    for (auto& obs : obstacles)                    // 3. obstacles
        obs.dessiner(fenetre);
    joueur.dessiner(fenetre);                      // 4. joueur
    // 5. HUD : vies, barre de progression, chrono
    fenetre.draw(barreProgressionFond);
    fenetre.draw(barreProgressionRemplie);
}
```

6.8 ArrierePlan::reinitialiser()

ArrierePlan::reinitialiser() --- remise à zéro au début d'un niveau

```
void ArrierePlan::reinitialiser() {
    if (!charge) return;
    for (int i = 0; i < 2; ++i)
        tuiles[i].setPosition(i * (float)Constantes::LARGEUR_FENETRE, 0.f);
}
```

7. Besoins de Compilation

7.1 Prérequis logiciels

Composant	Description
Compilateur C++	g++ ≥ 7 ou clang++ ≥ 5 --- Compile les sources .cpp en binaire exécutable
Standard C++	C++17 --- Requis pour les fonctionnalités modernes utilisées
SFML	2.5.x ou 2.6.x --- Bibliothèque graphique : rendu, événements, horloge
CMake (optionnel)	≥ 3.10 --- Outil de build multi-plateforme recommandé
pkg-config	Linux/macOS --- Détecte automatiquement les chemins SFML

7.2 Structure complète des fichiers du projet

Arborescence du projet

```
projet_sfrch/
├── main.cpp                          ← point d'entrée
├── constantes.h                      ← namespace Constantes
├── EtatJeu.h                         ← enum class EtatJeu
├── Jeu.h  / Jeu.cpp                  ← classe principale
├── Joueur.h / Joueur.cpp             ← personnage joueur
├── Obstacle.h / Obstacle.cpp         ← obstacles sol/aérien
├── ArrierePlan.h / ArrierePlan.cpp   ← fond défilant
```

```
├── TexturesObstacles.h / TexturesObstacles.cpp
├── perso.png
├── arriereA.jpg.jpeg  arriereB.jpg.jpeg  arriereC.jpg.jpeg
├── obstA.png  obst2A.png  obstB.png  obst2B.png
├── obstC.png  obst2C.png
└── arial.ttf
```

7.3 Installation de SFML selon la plateforme

Ubuntu / Debian / Linux Mint

```
sudo apt update
sudo apt install libsfml-dev
```

Plateforme	Commande
Arch Linux / Manjaro	sudo pacman -S sfml
Fedora / RHEL	sudo dnf install SFML-devel
macOS avec Homebrew	brew install sfml
Windows avec MSYS2	pacman -S mingw-w64-x86_64-sfml

Windows avec vcpkg

```
git clone https://github.com/microsoft/vcpkg
.\vcpkg\bootstrap-vcpkg.bat
.\vcpkg\vcpkg install sfml:x64-windows
```

7.4 Compilation manuelle avec g++

Commande de compilation complète

```
g++ -std=c++17 -Wall -Wextra -O2 \
    main.cpp Jeu.cpp Joueur.cpp Obstacle.cpp \
    ArrierePlan.cpp TexturesObstacles.cpp \
    -lsfml-graphics -lsfml-window -lsfml-system \
    -o jeu_sfrch
./jeu_sfrch
```

Build debug avec AddressSanitizer

```
g++ -std=c++17 -Wall -g -fsanitize=address \
    main.cpp Jeu.cpp Joueur.cpp Obstacle.cpp \
    ArrierePlan.cpp TexturesObstacles.cpp \
    -lsfml-graphics -lsfml-window -lsfml-system \
    -o jeu_sfrch_debug
./jeu_sfrch_debug
```

7.5 Compilation avec CMake (recommandé)

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(JeuSFRCH CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(MSVC)
    add_compile_options(/W4)
else()
    add_compile_options(-Wall -Wextra)
endif()
find_package(SFML 2.5 COMPONENTS graphics window system REQUIRED)
add_executable(jeu_sfrch
    main.cpp Jeu.cpp Joueur.cpp Obstacle.cpp
    ArrierePlan.cpp TexturesObstacles.cpp)
target_link_libraries(jeu_sfrch sfml-graphics sfml-window sfml-system)
```

Build avec CMake

```
mkdir build && cd build
cmake ..
make -j4
./jeu_sfrch
```

7.6 Drapeaux de compilation --- Référence

Drapeau	Description
-std=c++17	Active le standard C++17. Requis pour if constexpr, std::optional, structured bindings.
-Wall	Active la majorité des avertissements utiles.
-Wextra	Avertissements supplémentaires : paramètres inutilisés, comparaisons signées/non-signées.
-O2	Optimisation niveau 2 pour la release.
-O0 -g	Pas d'optimisation + symboles de debug. Nécessaire pour gdb / lldb.
-fsanitize=address	AddressSanitizer : détecte les dépassements de buffer, use-after-free, fuites mémoire.
-lsfml-graphics	Lie avec SFML Graphics (sf::Sprite, sf::Texture, sf::RenderWindow...).
-lsfml-window	Lie avec SFML Window (sf::Event, sf::Keyboard, gestion de fenêtre).
-lsfml-system	Lie avec SFML System (sf::Clock, sf::Vector2f, sf::Time).

7.8 Références

- **Ressource Vidéo :** *Formation Programmation C++* (YouTube)
    - Utilisée pour appréhender la logique globale et consolider les concepts de base du langage.
- **Documentation Officielle :** *Tutoriel SFML 3.0 — Gestion des fenêtres*
    - Source de référence principale pour l'implémentation technique et la maîtrise de la syntaxe spécifique à la bibliothèque graphique.
- **Support de Cours :** *Cours POO en C++* de **P. RACHIDA HANNANE**
    - Référence théorique essentielle pour l'application des concepts académiques de la Programmation Orientée Objet au sein du projet.
- **Crédits Visuels :** L'intégralité des illustrations et ressources graphiques a été sélectionnée via **Google Images**.

8. Conclusion

8.1 Bilan technique

Ce mini-projet nous a permis de mettre en pratique de manière concrète et cohérente les fondements de la Programmation Orientée Objet en C++. En partant d'une idée simple --- un jeu de runner 2D (SFRCH) --- nous avons conçu une architecture logicielle structurée, modulaire et extensible autour de six classes bien délimitées.

Chacun des quatre piliers de la POO a été mis en œuvre ou préparé dans ce projet. L'encapsulation est présente dans toutes les classes via des méthodes d'accès contrôlées. La séparation entre interface (.h) et implémentation (.cpp) matérialise parfaitement le principe d'abstraction. La règle des trois, implémentée dans **Joueur**, garantit la sécurité des copies d'objets contenant des ressources SFML.

8.2 Ce que nous avons appris

— La modélisation d'un problème réel sous forme de classes avec des responsabilités clairement définies.

— La règle des trois en C++ pour la gestion correcte des ressources SFML (textures, sprites).

— La gestion d'une boucle de jeu avec machine à états finis (EtatJeu) et séparation événements/logique/rendu.

— Le défilement infini avec recyclage de tuiles dans la classe ArrierePlan.

— L'importance des déclarations anticipées (forward declarations) pour éviter les dépendances circulaires.

— Le rendu 2D avec sf::Sprite, sf::Texture, sf::RectangleShape, sf::Text et leurs propriétés graphiques.

— La centralisation des constantes dans un namespace dédié pour une configuration unifiée du jeu.

8.4 Conclusion

Ce mini-projet, bien que modeste dans son apparence de jeu de runner 2D, s'est révélé être un terrain d'apprentissage remarquablement riche et structurant. En partant d'une idée simple — un personnage qui court, saute et esquive des obstacles — nous avons été amenés à mobiliser, articuler et approfondir un ensemble de concepts fondamentaux de la programmation orientée objet en C++, que ce soit l'encapsulation, l'abstraction, ou encore la gestion rigoureuse des ressources graphiques via la règle des trois.

La bibliothèque SFML nous a offert un cadre concret et immédiatement visuel pour ancrer ces notions théoriques dans une réalité tangible : chaque classe conçue avait un rôle précis, chaque méthode une responsabilité claire, et chaque décision d'architecture une conséquence directement observable à l'écran. Cette dimension pratique a considérablement renforcé notre compréhension, bien au-delà de ce qu'un exercice purement académique aurait pu nous apporter.

Au-delà du code lui-même, ce projet nous a appris à penser en termes de systèmes : comment une machine à états finis orchestre le déroulement d'un jeu, comment une boucle principale sépare proprement les événements, la logique et le rendu, ou encore comment centraliser les constantes pour rendre une base de code évolutive et maintenable.

En définitive, ce socle que nous avons construit n'est pas une fin en soi, mais bien un point de départ. Les améliorations envisagées — héritage, animations, effets sonores, sauvegarde des scores — sont autant de portes ouvertes vers une maîtrise toujours plus approfondie du C++ orienté objet et du développement de jeux 2D avec SFML.